Transport Layer Optimization for Real-Time Healthcare Telemetry in Congested Wireless Networks

[Author One]
School / Department
University / Organisation
City, Country
author.one@example.org

[Author Two]
School / Department
University / Organisation
City, Country
author.two@example.org

[Author Three]
School / Department
University / Organisation
City, Country
author.three@example.org

[Author Four / Corresponding Author]
School / Department
University / Organisation
City, Country
author.four@example.org

***Abstract*—Continuous patient monitoring over shared wireless networks produces small, periodic, deadline-bound traffic (alarms, ECG waveforms and vital signs) that competes with bulk transfers. We start from an ns-3 baseline in which TCP and UDP healthcare telemetry share an 802.11g access point with 1–8 Mbit/s of background traffic, and show that there neither protocol is hurt: healthcare packet loss is zero and delay stays below 2.5 ms, because the congestion stays inside the background sender's own queue, and TCP's higher measured throughput is header overhead. We validate a Python replica of that scenario against the ns-3 results (throughput within 1%, saturated-flow delay within 1%, healthcare delay only approximately) and evaluate a priority- and deadline-aware transport framework in a stress model with a shared access-point queue and bursty wireless loss. The framework combines per-class message streams, deadline-aware selective retransmission, loss-adaptive forward error correction with a delay-based congestion gate, and priority queueing with deadline-aware dropping. It is compared with TCP, UDP, UDP with priority queueing and a transport-only variant, using 8 replications per setting. With 40 patients and 8 background bulk flows, the proposed framework delivers 100.0% of alarms and 100.0% of ECG messages within their deadlines, against 20.1% and 72.1% for TCP, and keeps 99th-percentile ECG latency at 40 ms against 990 ms, at the cost of an overhead ratio of 1.28 and an 11% reduction in background goodput relative to priority queueing alone. The results are simulation-based and assume strict priority at the access point; testbed validation remains future work.**

***Index Terms*—healthcare telemetry, transport protocol, deadline-aware retransmission, forward error correction, priority queueing, network congestion, wireless networks.**

I. INTRODUCTION

Continuous patient monitoring is increasingly delivered over wireless local area networks. Bedside monitors, wearable ECG patches and pulse oximeters stream waveforms and vital signs to central stations and electronic health record systems, and raise alarms when a measured value crosses a clinical threshold [11]. These flows are small, periodic and *deadline-bound*: an alarm that arrives after a few hundred milliseconds, or an ECG segment that arrives after the display window has moved on, has little clinical value even if it is eventually delivered correctly. At the same time, the wireless network they share is rarely idle. Imaging uploads, software updates, guest traffic and other bulk transfers compete for the same access point, producing standing queues, bursty losses and variable delay.

The transport layer is where this tension is felt most directly. TCP [3] guarantees reliable, in-order delivery, which is the wrong guarantee for fresh telemetry: a single lost segment stalls every later message on the connection (head-of-line blocking), and loss-based congestion control such as CUBIC [7] fills the bottleneck buffer until packets are dropped, inflating queueing delay for everyone [13]. Many healthcare IoT deployments nevertheless ride on TCP through application protocols such as MQTT, whose delivery guarantees add latency of their own [9]. UDP avoids these effects but offers no recovery at all, so every wireless loss becomes a missing sample or a missed alarm.

This paper starts from a baseline experiment, an ns-3 simulation of TCP and UDP healthcare telemetry competing with background traffic on an 802.11g network [1, 2], and extends it into a study of the transport layer for real-time telemetry under congestion. We then propose a priority- and deadline-aware transport framework that treats the *deadline* of each message, not reliability in the abstract, as the quantity to protect. Our contributions are:

- an analysis of the baseline ns-3 experiment, showing why TCP and UDP look equivalent there (the congestion is confined to the sender that causes it) and why TCP's higher "throughput" is header overhead;
- a Python replica of the 802.11g scenario, validated against the ns-3 results, and an open, interactive Python model that the paper's numbers can be regenerated from;
- a message model with three telemetry classes (alarm, ECG waveform, vital signs), each with a delivery deadline, plus a best-effort bulk class;
- deadline-aware selective retransmission with a short, measured timer, which retransmits only when the repair can still arrive in time;

- loss-adaptive forward error correction for the waveform stream and redundant copies for alarms, together with a delay-based *congestion gate* that switches redundancy off when queueing, rather than channel loss, is the cause of delay;
- class-based priority queueing at the access point with deadline-aware dropping of messages that can no longer arrive in time; and
- a reproducible discrete-time study comparing five transport configurations across congestion level, patient population, channel loss and buffer size.

All results come from simulation of stylised network models; they are not measurements from a clinical deployment. Section V states the assumptions and limitations explicitly.

The remainder of the paper is organised as follows. Section II reviews related work. Section III defines the system model and metrics. Section IV presents the proposed framework. Section V describes the implementation, the baseline and validation results, and the stress-scenario results, and Section VI concludes.

II. RELATED WORK

1) Networking for healthcare monitoring. A recent survey of the Internet of Medical Things reviews the wireless links used to carry patient data, from body-area networks to backhaul, and lists delay, interference and reliability among the open challenges [11]. Experimental work on remote patient monitoring over MQTT shows that protocol choices change message latency measurably: for small messages typical of vital signs, a lightweight cipher outperformed AES-based security in round-trip time [10], and MQTT's highest delivery-guarantee level was reported to raise delivery latency to more than seven times the average network delay [9]. These studies motivate treating delay as a design target for the transport, but they do not address congestion at a shared access point.

2) Transport protocols and loss recovery. The TCP specification requires in-order delivery to the application [3]. Time-based loss detection with tail-loss probes (RACK-TLP) [6] lets TCP detect losses and avoid retransmission timeouts sooner, and QUIC [4,5] multiplexes independent streams over UDP so that loss on one stream does not delay the others. These mechanisms narrow the gap to a deadline-aware design but remain deadline-agnostic: they retransmit whether or not the repair can still arrive in time. A recent analysis of deadline-constrained traffic in random-access networks formalises this trade-off, showing that allowing more retransmissions improves reliability but adds queueing delay that can cause packets to expire [12]. Our retransmission rule is a practical instance of the same idea: retransmit only if the repair can still meet the deadline, and cap the number of attempts.

3) Queue management and quality of service. Large buffers combined with loss-based congestion control produce persistent queueing delay. Field measurements of cable modems found upstream delay of roughly 15–30 ms with active queue management (AQM) versus about 250 ms without it [13], and an emulated Wi-Fi study of 14 congestion-control algorithms and 7 AQM schemes found the COBALT AQM gave the lowest delay across all of them [14]. The L4S architecture pursues low queueing delay through ECN-based signalling and new sender congestion controls [8]. For class-based service, an analytical model of IEEE 802.11e EDCA reports considerably lower delay and jitter for the high-priority access categories under saturated, ideal-channel conditions [15]. Our framework assumes class-based prioritisation is available at the access point and studies what the transport layer should do on top of it.

TABLE I
TRAFFIC CLASSES

Class	Stream	Payload	Rate / patient	D_c	Priority
0	Alarm / event	60 B	Poisson, mean 5 s	150 ms	highest
1	ECG waveform	200 B	1 per 40 ms	300 ms	high
2	Vital signs	80 B	1 per 1 s	1000 ms	medium
3	Bulk (cross traffic)	1500 B	greedy TCP	none	lowest

4) Simulation methodology. Validating a network simulator against an independent model is established practice; for example, ns-3's 802.11ax OFDMA implementation has been compared against an analytical contention model [16]. We follow the same spirit by checking our Python Wi-Fi replica against the ns-3 results of the baseline repository [1].

5) Gap. Among the sources cited here, we did not find an evaluation that combines deadline-classified clinical message streams, loss-adaptive redundancy gated by a delay signal, and deadline-aware queue dropping against TCP and UDP baselines under congestion. This is not a claim that none exists; a systematic literature search is outstanding.

III. SYSTEM MODEL AND PROBLEM STATEMENT

A. Network model

N patient-worn or bedside devices send telemetry over a shared wireless uplink through one access point (AP) to a monitoring server. The AP's transmit queue in front of the wireless link is the only bottleneck. The link serves C_g bytes per millisecond in the *good* channel state and $C_b < C_g$ in the *bad* state. State changes follow a two-state Gilbert–Elliott burst-error process, and each transmitted packet is lost with probability ℓ_g or ℓ_b according to the current state. K greedy bulk TCP flows share the same queue as cross traffic. A fixed one-way delay d models the remaining path, and the return path is assumed lossless.

B. Message classes and deadlines

Every telemetry message m belongs to a class $c \in \{0,1,2\}$, is generated at time g_m and must reach the receiver by $g_m + D_c$. Class 3 is best-effort bulk traffic without a deadline. Table I lists the classes used in this study. The payload sizes, rates and deadlines are design assumptions for evaluation, not clinical standards.

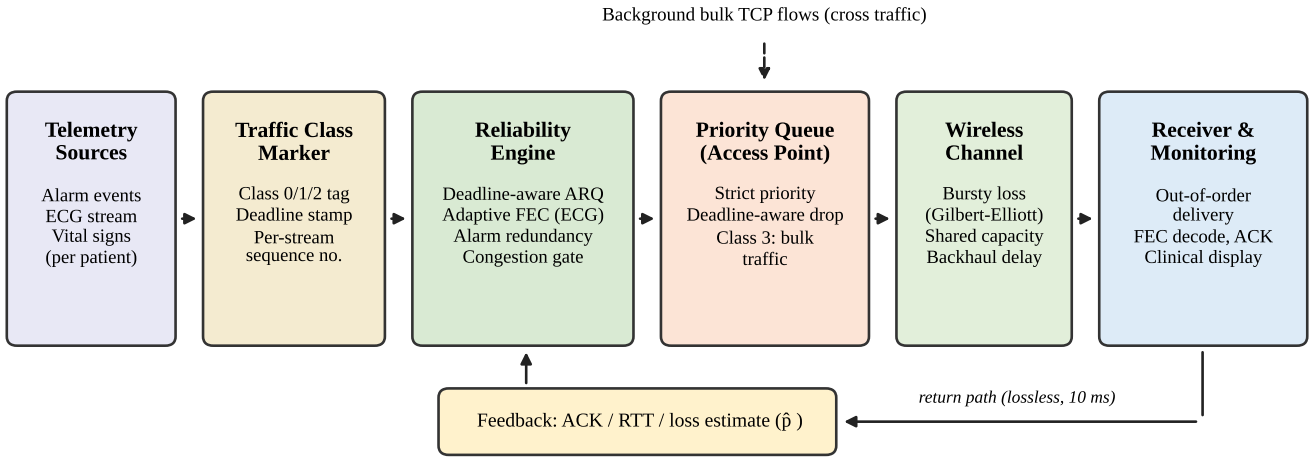


Fig. 1. Architecture of the proposed priority- and deadline-aware transport framework. Solid arrows show the data path; the feedback block supplies RTT and loss estimates that drive retransmission timing, adaptive FEC and the congestion gate.

C. Metrics

The primary metric is the *timely delivery ratio* of class c,

$$\mathrm{TDR}_c = \frac{\left|\{m \in \mathscr{M}_c : t_m^{\mathrm{del}} \le g_m + D_c\}\right|}{|\mathscr{M}_c|}, \tag{1}$$

where t_m^{del} is the time the message is handed to the receiving application (for TCP, after in-order reassembly) and undelivered messages count as misses. Secondary metrics are the 99th-percentile latency of delivered messages, the *overhead ratio* O (all telemetry bytes offered to the link, including headers, retransmissions and parity, divided by the bytes of first transmissions), and the goodput obtained by the background bulk flows, which measures what the telemetry protection costs other users.

D. Problem statement

Given N, K, the channel parameters and the buffer size, choose a transport behaviour that maximises TDR_c for the telemetry classes while keeping O small and not starving class 3.

IV. Proposed Methodology

The proposed framework is a chain of cooperating modules, shown in Fig. 1. Unlike the one-connection-per-device TCP baseline, each telemetry class forms an independent message stream per patient with its own sequence numbers, so a loss in one stream never delays another.

A. System Modules

1) Traffic class marker. Each message is tagged with its class, a per-stream sequence number and an absolute deadline $g_m + D_c$. The class selects the AP queue (for example through a DiffServ code point or an 802.11e access category [15]) and the reliability treatment. The deadline travels in the packet header so that the AP can apply deadline-aware dropping.

2) Deadline-aware selective ARQ. Each stream keeps a smoothed round-trip time srtt_s and variation rttvar_s, updated from acknowledgements of first transmissions only (Karn's rule). A message that is not acknowledged within

$$\mathrm{RTO}_s = \max\left(30\ \mathrm{ms},\ \mathrm{srtt}_s + 4\,\mathrm{rttvar}_s\right) \tag{2}$$

is retransmitted if and only if the repair can still arrive in time and the retry budget is not exhausted:

$$r < R_{\max} \quad \wedge \quad t + \tfrac{1}{2}\mathrm{srtt}_s \le g_m + D_c, \tag{3}$$

with $R_{\max} = 3$ and successive timers scaled by 1.5^r. Otherwise the message is abandoned, which frees capacity for fresher data. The 30 ms floor, compared with 200 ms for TCP, is what allows a repair to arrive inside a 150 ms alarm deadline.

3) Adaptive FEC and alarm redundancy. For each patient a loss indicator $\hat{p}$ is maintained as an exponentially weighted average, $\hat{p} \leftarrow (1-\alpha)\hat{p} + \alpha\lambda$ with $\alpha = 0.02$, where $\lambda = 1$ when a retransmission timer fires and $\lambda = 0$ when a first transmission is acknowledged. ECG packets are protected in groups by one XOR parity packet, which lets the receiver rebuild any single lost packet of the group without a round trip. The group size follows $\hat{p}$:

$$g(\hat{p}) = \begin{cases} 0\ (\text{no FEC}) & \hat{p} < 0.01 \\ 4 & 0.01 \le \hat{p} < 0.06 \\ 2 & \hat{p} \ge 0.06. \end{cases} \tag{4}$$

Alarm messages are single events, so when $\hat{p} > 0.01$ a second copy is sent 10 ms after the first, which gives time diversity against short loss bursts.

4) Congestion gate. Redundancy helps against channel loss but only adds load when delay is caused by queueing. The gate therefore disables FEC and alarm copies for a patient while the ECG stream's smoothed RTT exceeds $1.5\,(2d + 4\ \mathrm{ms})$, i.e. when a standing queue is visible in the RTT. Section V-H shows that omitting this gate causes the protection traffic itself to overload the link at high patient counts.

Algorithm 1 Sender-side message handling

Require: Message m of class c, patient p, time t
Ensure: Packets handed to the AP queue

Tag m with class, sequence number, deadline $g_m + D_c$
open $\leftarrow (\text{srtt}_{p,\text{ECG}} \leq 1.5(2d+4))$ {congestion gate}
Send m; arm timer RTO_s
if $c = \text{ECG}$ **and** open **and** $\hat{p}_p \geq 0.01$ **then**
    Add m to current XOR group of size $g(\hat{p}_p)$
    if group is full **then**
        Send parity packet
    end if
else if $c = \text{Alarm}$ **and** open **and** $\hat{p}_p > 0.01$ **then**
    Send a second copy after 10 ms if m is still unacknowledged
end if
on timer expiry: update $\hat{p}_p$; **if** $r < R_{\max}$ **and** $t + \frac{1}{2}\text{srtt} \leq g_m + D_c$ **then** retransmit and re-arm **else** abandon
on acknowledgement: cancel timer; update srtt, rttvar, $\hat{p}_p$

Algorithm 2 Access-point queue service

Require: Class queues $Q_0, \ldots, Q_3$, link capacity C_t at time t
Ensure: Transmitted packets

Add C_t bytes of transmission credit
while some queue is non-empty **do**
    $h \leftarrow$ head of the highest-priority non-empty queue
    if h is telemetry **and** $t + d > \text{deadline}(h)$ **then**
        Drop h {cannot arrive in time}
    else if credit $\geq |h|$ **then**
        Transmit h; deduct $|h|$ from credit
    else
        break
    end if
end while

5) Priority queue with deadline-aware dropping. The AP serves four class queues in strict priority order (alarm, ECG, vitals, bulk), with per-class buffer limits so that bulk traffic cannot occupy telemetry buffer space. At dequeue time t, a telemetry packet is discarded without being transmitted if $t + d > g_m + D_c$, since it can no longer arrive on time. Such packets would otherwise consume wireless capacity that fresher messages need.

6) Receiver. The receiver delivers each message to the application as soon as it arrives, without in-order reassembly, acknowledges every copy it receives, rebuilds a lost ECG packet when the parity packet and all other group members are present, and acknowledges recovered packets so that the sender cancels their timers.

B. Algorithms

Algorithm 1 describes what the end host does for each message and Algorithm 2 describes the queue discipline at the access point.

TABLE II
DEFAULT SIMULATION PARAMETERS

Parameter	Value
Link capacity, good / bad state	8.0 / 4.8 Mbit/s
Mean sojourn, good / bad state	700 ms / 40 ms
Packet loss, good / bad state	0.5% / 25% (ACKs equally affected)
One-way delay d	10 ms
FIFO buffer (baselines)	200 kB
Priority-queue buffers (alarm / ECG / vitals / bulk)	10 / 30 / 10 / 200 kB
Monitored patients N	40
Background bulk TCP flows K	8
Packet header (all transports)	40 B
TCP	Reno-style, initial window 10, SACK-based recovery, min. RTO 200 ms
Simulated time	65 s (5 s warm-up, last 3 s excluded)
Replications	8 seeds per setting, 95% t-intervals

V. IMPLEMENTATION AND RESULTS

A. Implementation

The study uses three levels of model, all provided with this paper so that every number can be regenerated. *Level 1 is the baseline ns-3 experiment* of the base repository [1]: an ns-3.47 [2] simulation with one healthcare station (80 kbit/s of 100 B packets, over UDP or TCP) and one background station (constant-rate UDP, 1000 B packets, 1, 4 or 8 Mbit/s) sending to an access point over IEEE 802.11g at a fixed 6 Mbit/s rate, with applications active from 2 s to 9 s of a 15 s run. We did not re-run ns-3 for this paper (the ns-3 distribution could not be built in our environment); we use the repository's published results file unchanged. *Level 2 is a Python replica of that scenario*: a slot-level 802.11g distributed coordination function (DCF) model with carrier sensing, binary exponential backoff, collisions, MAC acknowledgements, a 500-packet/500 ms sender queue and, for TCP, delayed acknowledgements returned by the access point. It models neither TCP congestion dynamics (the flow is application-limited) nor channel errors; in the ns-3 scenario all nodes share one position, so the channel is lossless. Its metrics follow ns-3 FlowMonitor definitions (IP-level bytes, mean delay, mean absolute delay difference as jitter). *Level 3 is the stress model* (a discrete-time simulator written in Python with NumPy, SciPy and Matplotlib for analysis) of a shared access-point bottleneck: it advances in 1 ms steps and models the AP queue, a Gilbert–Elliott wireless channel, a fixed backhaul delay, TCP senders and receivers, and the proposed transport. This is the scenario in which the proposed framework is evaluated, because, as Section V-C shows, the baseline does not stress the transport. Table II lists its parameters. MAC-layer contention among stations, rate adaptation and per-device radio conditions are not modelled at this level; channel state is shared by all devices and is treated as a property of the access point's wireless medium.

B. Compared Configurations

Five configurations are compared (Table III). *TCP* models MQTT-over-TCP deployments: one connection per patient carrying all three telemetry classes, in-order delivery and a shared FIFO bottleneck queue. *UDP* sends each message as one data-

TABLE III
CONFIGURATIONS AND MECHANISMS

	Recovery	Order	Prio. queue	Dl. drop	FEC/copies
TCP	ARQ (TCP)	in-order	–	–	–
UDP	none	none	–	–	–
UDP+Prio	none	none	✓	–	–
Transport-only	dl.-aware ARQ	none	–	–	✓
Proposed	dl.-aware ARQ	none	✓	✓	✓

gram with no recovery. *UDP+Prio* adds class-based priority queueing at the AP and isolates the benefit of prioritisation alone. *Transport-only* runs the proposed end-host mechanisms (deadline-aware ARQ, adaptive FEC, alarm copies, congestion gate) over a plain FIFO queue and isolates the benefit of the transport logic without network support. *Proposed* adds priority queueing and deadline-aware dropping. Because the configurations build on one another, the comparison doubles as an ablation study.

C. Baseline Experiment (ns-3) and Validation

1) Baseline results. Fig. 2 reproduces the repository's five result graphs [1]. Healthcare packet loss is zero for both protocols at every background load (Fig. 2(e)), and healthcare throughput is flat (Fig. 2(a)). Healthcare delay stays between 0.70 ms (UDP) and 0.54 ms (TCP) at 1 Mbit/s and 2.42 ms and 2.40 ms at 8 Mbit/s (Fig. 2(c)), roughly two orders of magnitude below the 150 ms alarm deadline assumed in this paper; jitter stays below 1.9 ms (Fig. 2(d)). The only metric that degrades is that of the background flow itself: at 8 Mbit/s, which exceeds the roughly 4.8 Mbit/s that the 6 Mbit/s link can carry for 1000 B packets, the background flow's mean delay is 464 ms (UDP) and 465 ms (TCP) and the repository reports 949 and 982 lost background packets (Fig. 2(b)). Note that 6999 background packets were offered and 4411 (UDP) and 4338 (TCP) were received, so the number of undelivered packets (about 2588–2661) is larger than the reported loss count; we do not know from the repository how the remaining packets were accounted for, and our replica counts them as undelivered.

2) TCP versus UDP throughput. TCP's higher healthcare throughput in Fig. 2(a) (121.5 against 102.7 kbit/s) is not a protocol advantage. Both flows offer the same 80 kbit/s of 100 B payloads, 100 packets per second, and FlowMonitor counts IP-level bytes: a UDP packet is 100+8+20=128 B and a TCP packet with its header options is 152 B, which gives exactly $100 \times 128 \times 8 = 102.4$ and $100 \times 152 \times 8 = 121.6$ kbit/s. The difference is header overhead.

3) Why the baseline does not stress the transport. The healthcare and background flows originate at different stations, each with its own sender queue. The congestion at 8 Mbit/s is therefore confined to the background station's own queue (hundreds of milliseconds of queueing), whereas the healthcare station contends for the channel separately: with IEEE 802.11 distributed coordination, a light flow waits at most for the frame in flight (about 1.5 ms at 6 Mbit/s for a 1000 B packet) plus a short backoff. This is consistent with the 2–3 ms healthcare delays at 8 Mbit/s, and it is reproduced by our replica, which has the same structure. In this topology neither TCP nor UDP is hurt by the congestion, so the baseline cannot show any benefit from a transport optimisation. The remaining limitations of the baseline are that each point is a single run, with both applications starting at exactly the same simulated time, that only three load levels are used, that the channel is lossless (all nodes share one position) and that there is one healthcare device. We therefore treat the baseline as a validated starting point and evaluate the proposed framework in a scenario in which telemetry shares a bottleneck queue with bulk traffic (Section V-D onwards).

4) Validation of the Python replica. Fig. 3 and Table IV compare the replica (30 runs per setting) with the single ns-3 run per setting. Throughput agrees within 0.6% for both flows at all loads, and the saturated background flow's delay at 8 Mbit/s agrees within 0.7%. Healthcare delay at 8 Mbit/s is 2.76 ms (UDP) and 2.86 ms (TCP) against 2.42 and 2.40 ms in ns-3, an overestimate of 14–19%. At 1 Mbit/s the replica overestimates healthcare delay (about 0.88 ms against 0.70 ms for UDP and 0.91 ms against 0.54 ms for TCP), and at 4 Mbit/s, where the offered load is about 80% of capacity, its mean delay is heavy-tailed (8.8 ms for UDP and 6.2 ms for TCP, with medians of 3.1 and 3.0 ms) while the ns-3 run gives 1.96 and 0.55 ms. We therefore regard the replica as validated for throughput, background delay and the qualitative result (no healthcare loss and millisecond-scale delay), and as only approximately validated for healthcare delay and jitter, which a single deterministic ns-3 run cannot pin down; the ns-3 results could not be re-run with other seeds in our environment.

D. Stress-Scenario Metrics and Statistical Treatment

Each setting was simulated with 8 independent seeds. For a given seed all configurations see the same message schedule (patient phases and alarm times), so differences between configurations are not caused by different workloads. Tables report the mean and the half-width of a 95% Student-t confidence interval over the 8 runs; figures show the same intervals as error bars. A reported timely delivery ratio of 100.0% means that fewer than 0.05% of messages missed their deadline on average, not that a miss is impossible. Messages generated in the first 5 s and last 3 s of each run are excluded.

E. Results in the Default Congested Scenario

Table V compares the five configurations with 40 patients and 8 background bulk flows. TCP delivers only 20.1% of alarms and 72.1% of ECG messages by their deadlines, and the 99th-percentile latency of its alarms is about 1052 ms. The shared 200 kB buffer is the main cause: when the eight bulk flows keep it full, it takes about 200 ms to drain at the good-state rate, which alone exceeds the 150 ms alarm deadline, and TCP adds head-of-line blocking and retransmission-timer back-off on top of the queueing delay. UDP avoids the blocking and keeps ECG at 98.5%, because the 300 ms ECG deadline is longer than

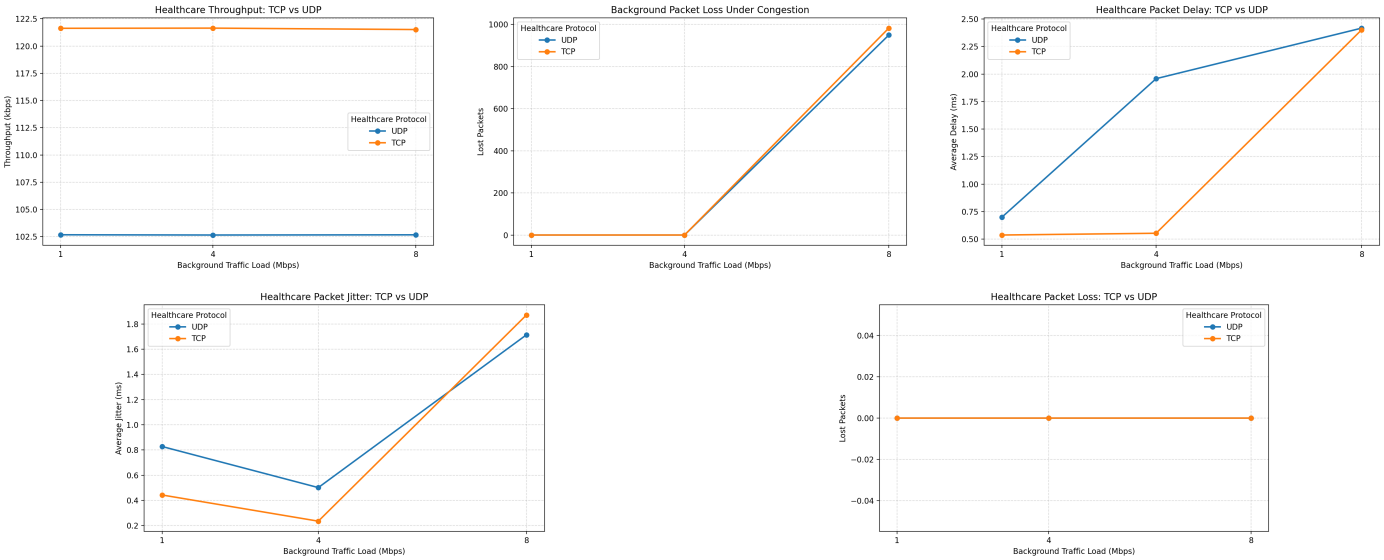


Fig. 2. Baseline ns-3 results from the repository [1] (single run per point): (a) healthcare throughput, (b) background packet loss, (c) healthcare delay, (d) healthcare jitter, (e) healthcare packet loss, for TCP and UDP at 1, 4 and 8 Mbit/s of background load.

TABLE IV
VALIDATION: NS-3 (REPOSITORY, ONE RUN) VERSUS PYTHON REPLICA (MEAN±95% CI, 30 RUNS). DELAY AND JITTER IN MS; BACKGROUND THROUGHPUT IN KBIT/S

	Load	Healthcare delay		Healthcare jitter		Background thr.		Background delay	
Proto.	(Mbit/s)	ns-3	replica	ns-3	replica	ns-3	replica	ns-3	replica
UDP	1	0.70	0.88±0.01	0.83	1.26±0.01	1029	1029±0	1.5	1.8±0.0
UDP	4	1.96	8.79±3.63	0.50	2.85±0.22	4114	4114±0	1.6	2.4±0.0
UDP	8	2.42	2.76±0.10	1.71	2.17±0.07	4835	4861±4	464	461±0
TCP	1	0.54	0.91±0.01	0.44	1.26±0.01	1029	1029±0	1.5	1.8±0.0
TCP	4	0.55	6.15±2.95	0.23	2.84±0.17	4114	4114±0	1.6	2.5±0.1
TCP	8	2.40	2.86±0.12	1.87	2.26±0.06	4756	4742±4	465	463±0

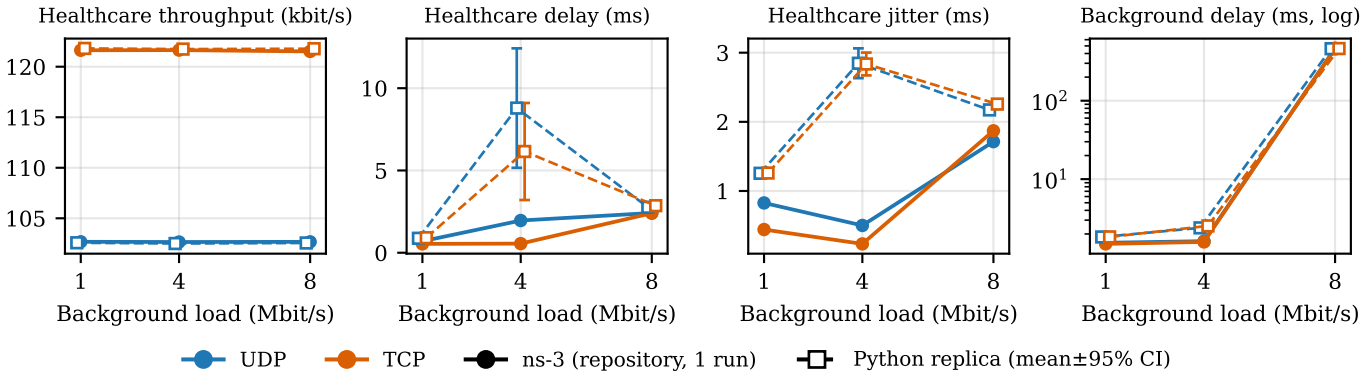


Fig. 3. Validation of the Python 802.11g replica against the repository's ns-3 results. Filled markers: ns-3 (one run). Open markers: replica, mean and 95% confidence interval over 30 runs. Markers are offset horizontally for readability.

the queueing delay, but only 31.4% of alarms arrive in time. Transport-only behaves like UDP for alarms (31.3%): no end-host mechanism can repair a message that is already late because it waited in a long queue. It does recover the losses that UDP cannot, bringing vitals to 100.0%.

UDP+Prio shows what prioritisation alone achieves: all three classes reach about 98% with latency equal to the path delay, but the remaining roughly 2% are messages lost on the wireless channel and never repaired. The proposed configuration combines both effects. It delivers 100.0% of alarms, 100.0% of ECG messages and 100.0% of vitals on time, with 99th-percentile latencies of 20 ms (alarms) and 40 ms (ECG). The price is an

TABLE V
RESULTS IN THE DEFAULT CONGESTED SCENARIO ($N = 40$ PATIENTS, $K = 8$ BULK FLOWS, 200 KB BUFFER); MEAN±95% CI OVER 8 RUNS

	Timely delivery ratio (%)			99th-pct latency (ms)		Overhead	Bulk goodput
Configuration	Alarm	ECG	Vitals	Alarm	ECG	ratio O	(Mbit/s)
TCP	20.1±4.2	72.1±2.6	99.0±0.4	1052±149	990±76	1.03±0.00	5.51±0.03
UDP	31.4±5.2	98.5±0.1	98.6±0.1	230±5	230±3	1.00±0.00	5.57±0.04
UDP+Prio	98.1±0.6	98.1±0.2	97.9±0.5	10±0	10±0	1.00±0.00	5.47±0.03
Transport-only	31.3±2.5	98.3±0.1	100.0±0.0	232±8	241±7	1.05±0.00	5.48±0.04
Proposed	100.0±0.0	100.0±0.0	100.0±0.0	20±8	40±0	1.28±0.03	4.86±0.08

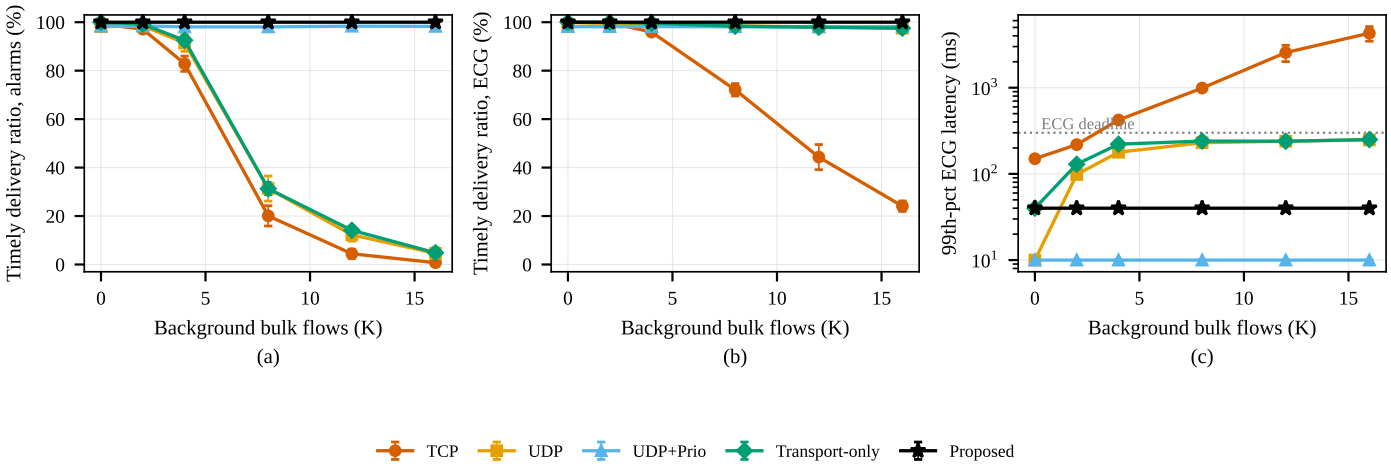


Fig. 4. Effect of background congestion ($N = 40$, 200 kB buffer): (a) timely delivery ratio of alarms, (b) of ECG messages, and (c) 99th-percentile ECG latency (log scale; dotted line: ECG deadline). Error bars are 95% confidence intervals over 8 runs.

overhead ratio of 1.28 and a background goodput of 4.86 Mbit/s instead of 5.47 Mbit/s for UDP+Prio, a reduction of 11%.

F. Effect of Congestion Level

Fig. 4 varies the number of background bulk flows. Without background traffic ($K = 0$) TCP is a reasonable choice: it delivers 99.2% of alarms and 99.9% of ECG messages on time. The advantage disappears quickly as congestion grows. At $K = 4$ TCP's alarm ratio is 82.9%, at $K = 8$ it is 20.1%, and at $K = 16$ it is 0.7%, with ECG at 24.1% and even vitals, whose deadline is 1 s, at 67.1%. UDP and Transport-only keep ECG above 97% but lose almost all alarms at $K = 16$ (4.6% and 4.8%). UDP+Prio is flat at about 98%. The proposed configuration stays at 100.0% for alarms and 100.0% for ECG, and its 99th-percentile latencies do not depend on K (alarms 19–21 ms, ECG 40 ms), because telemetry never waits behind bulk traffic.

G. Latency Distributions

Fig. 5 shows empirical latency distributions of delivered messages in the default scenario (pooled over three runs). TCP's distribution has a long tail beyond one second, which is the signature of head-of-line blocking after a loss and of retransmission-timer back-off. UDP and Transport-only are bounded by the queue drain time and cluster near 200 ms. UDP+Prio and the proposed configuration sit just above the 10 ms path delay; the proposed configuration has a short second mode at 30–40 ms, which corresponds to messages repaired by one retransmission. The CDFs cover delivered messages only, so they do not show the messages that were never delivered; the timely delivery ratios in Table V include those.

H. Scalability and the Role of the Congestion Gate

Fig. 6 increases the number of monitored patients from 20 to 120 with $K = 8$. Up to $N = 100$ the proposed configuration keeps every class at or above 99.9% while TCP falls from 30.9% to 0.3% of alarms. At $N = 120$ the telemetry itself, together with parity and retransmissions, approaches the capacity of the link; the proposed configuration still delivers 100.0% of alarms and 99.9% of ECG messages, but strict priority leaves almost nothing for the lowest telemetry class and for bulk traffic: vitals fall to 85.8% and background goodput to 0.08 Mbit/s.

Table VI shows why the congestion gate is needed. Without it, the loss indicator rises under load, FEC and alarm copies are switched on, and the extra traffic deepens the overload: at $N = 120$ the overhead ratio reaches 1.85 and vitals are delivered on time in 0.0% of cases. With the gate, the overhead stays at 1.40. An earlier version of the design without this gate collapsed in the same way during development, which motivated adding it.

I. Sensitivity to Channel Loss and Buffer Size

Fig. 7(a,b) varies the loss probability in the bad channel state from 5% to 55% at $K = 4$. TCP, UDP and Transport-only are limited mainly by queueing at this congestion level, so their alarm and ECG ratios change little with channel loss; for UDP

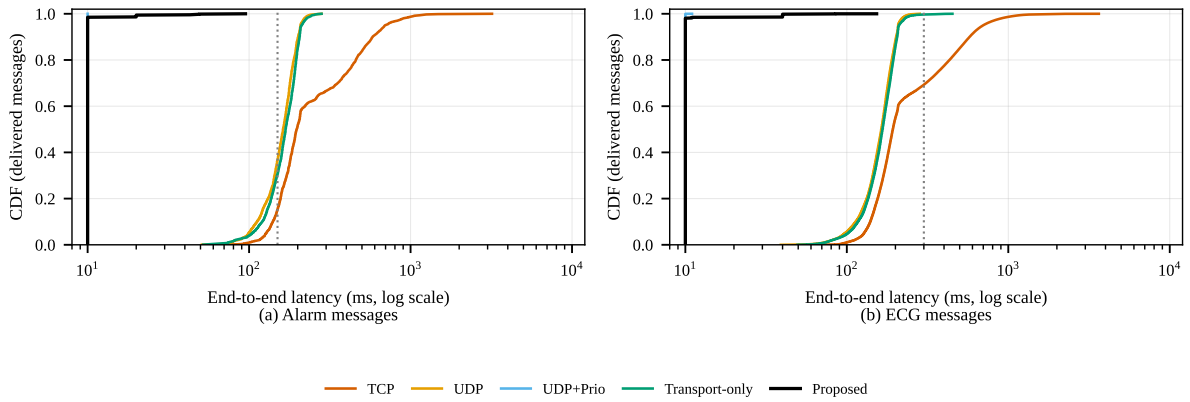


Fig. 5. Latency CDF of delivered messages in the default scenario for (a) alarms and (b) ECG messages (dotted lines: deadlines).

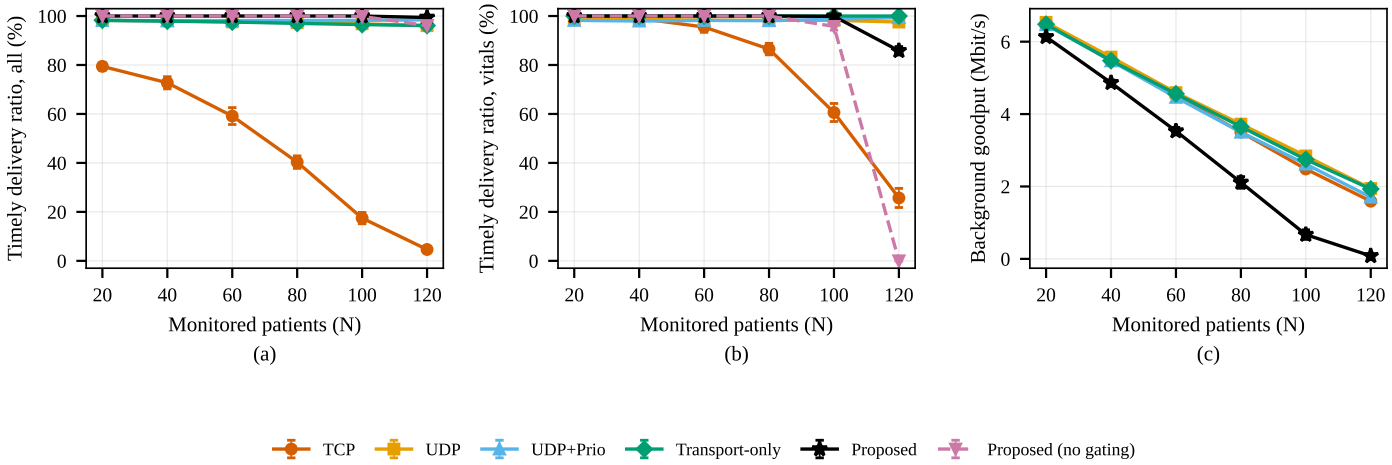


Fig. 6. Effect of the number of monitored patients ($K = 8$): (a) overall timely delivery ratio, (b) timely delivery ratio of vitals, which have the lowest telemetry priority, and (c) goodput of the background bulk flows.

TABLE VI
EFFECT OF THE CONGESTION GATE AT HIGH PATIENT COUNTS ($K = 8$)

N	Variant	Vitals TDR (%)	ECG p99 (ms)	O	Bulk (Mbit/s)
80	with gate	100.0±0.0	57±12	1.37±0.04	2.11±0.16
80	without gate	100.0±0.0	57±8	1.37±0.02	2.06±0.09
100	with gate	99.9±0.1	105±4	1.47±0.03	0.67±0.13
100	without gate	95.8±1.5	109±7	1.51±0.02	0.54±0.09
120	with gate	85.8±1.7	129±9	1.40±0.01	0.08±0.01
120	without gate	0.0±0.0	164±11	1.85±0.02	0.00±0.00

the alarm ratio even rises from 85.6% to 93.9%, most likely because more channel loss makes the bulk TCP flows back off and shortens the standing queue (we did not measure queue occupancy separately). UDP+Prio degrades steadily with channel loss (alarms from 99.1% to 96.3%) because it never repairs a loss. The proposed configuration stays at or above 99.9% for alarms and 100.0% for ECG at 55% bad-state loss, at the price of an overhead ratio that grows from 1.15 to 1.43.

Fig. 7(c) varies the bottleneck buffer for the FIFO configurations at $K = 8$. The result is a caution against over-selling transport changes: with a 25 kB buffer, which bounds queueing delay to about 25 ms, TCP delivers 96.2% of alarms on time and UDP 98.1%, and the case for a new transport is much weaker. The problem is largest for deep buffers: at 400 kB TCP delivers 16.7% of alarms. The proposed configuration is insensitive to buffer depth because telemetry has its own queues. Short buffers or active queue management [13, 14] are therefore an alternative or complement to prioritisation when it is not available at the access point.

J. Backhaul Delay and the Value of Redundancy

With a one-way delay of 10 ms a retransmission fits comfortably inside every deadline. Fig. 8 raises d to 120 ms ($N = 40$, $K = 8$) and adds an ARQ-only variant of the proposed configuration, which disables parity packets and alarm copies. At $d = 10$ ms ARQ alone already delivers 100.0% of alarms, ECG messages and vitals on time at an overhead ratio of 1.04, against 1.28 for the full design. Parity and alarm copies therefore cost about 24 points of overhead and bring no gain in timely delivery at short round-trip times; they only reduce the 99th-percentile alarm latency from 46 to 20 ms.

The picture changes as the round trip approaches the dead-

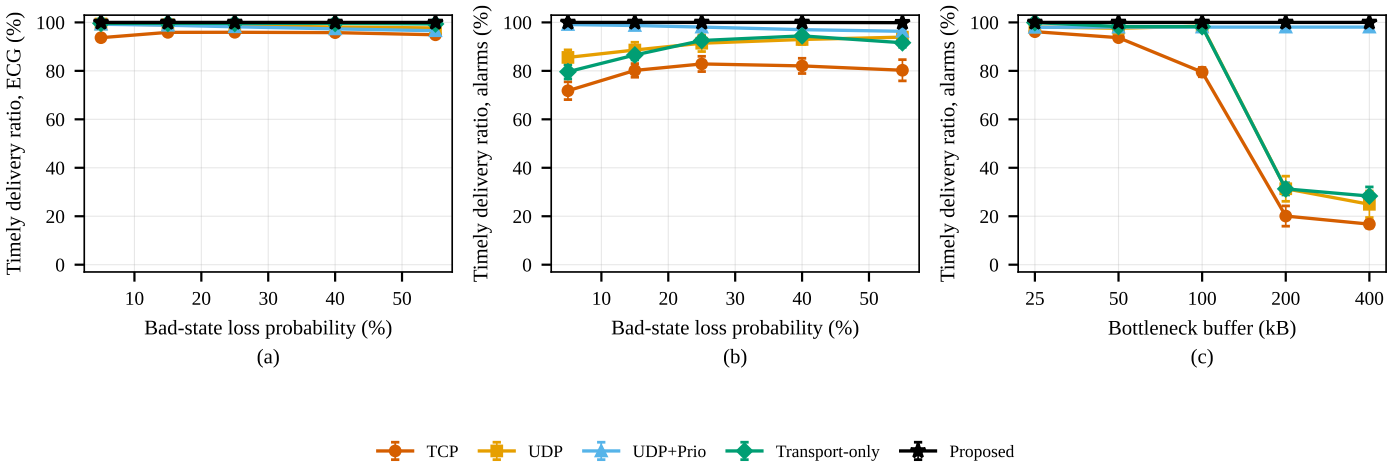


Fig. 7. Sensitivity analysis: (a) ECG and (b) alarm timely delivery ratio versus bad-state loss probability ($N = 40$, $K = 4$); (c) alarm timely delivery ratio versus bottleneck buffer size ($N = 40$, $K = 8$; the buffer of the telemetry queues in the priority configurations is fixed).

line. At $d = 80$ ms the full design delivers 99.7% of alarms on time against 98.4% for ARQ-only, and its 99th-percentile ECG latency is 120 ms instead of 240 ms. At $d = 120$ ms the retransmission rule rarely allows a repair to arrive in time, so ARQ-only matches UDP+Prio (alarms 98.0%, ECG 98.2%), while parity and copies still lift alarms to 99.6% and ECG to 99.7%. These gains are modest in absolute terms and cost an overhead ratio of 1.49. Redundancy is thus worth its cost mainly when the path delay is a substantial fraction of the deadline, and an implementation should enable it on that condition rather than on the loss estimate alone. TCP is far outside its useful range at this delay, with 11.3% of alarms on time.

At intermediate delays the overhead of both variants rises sharply (1.71 for the full design at $d = 40$ ms, with 21368 retransmissions per run against 2421 at $d = 10$ ms). The retransmission timer is srtt $+ 4$ rttvar with a small variance estimate, which leaves little margin for the delay jitter caused by bad-state periods; many retransmissions are then spurious. A margin proportional to srtt would probably reduce this waste, but we did not evaluate it, and this is a limitation of the present timer design.

K. Overhead and Cost to Background Traffic

At the default delay ($d = 10$ ms) the proposed configuration sends 2421 retransmissions, 16210 parity packets and 449 alarm copies per run, giving the overhead ratio of 1.28 in Table V. Parity packets recover only 238 messages per run, because the 30 ms retransmission timer usually repairs a loss first. The ARQ-only variant (Section V-J) shows the consequence: the same timely delivery at a much lower overhead when the round-trip time is short. The bandwidth taken from background flows is the cost of telemetry protection; at the default load it amounts to 11% of the goodput that UDP+Prio leaves them. Under strict priority, bulk traffic has no protection if telemetry load approaches capacity, as the $N = 120$ case shows; a production deployment would need a rate cap or weighted scheduling for the telemetry classes.

L. Discussion and Threats to Validity

1) Simulation, not measurement. All results come from a simplified model. Per-station radio conditions, MAC-layer contention and retries, rate adaptation and downlink traffic are not modelled; one channel state is shared by all devices; losses within a channel state are independent; and the access point is assumed to implement strict priority between classes. In real 802.11 networks, enhanced distributed channel access [15] gives statistical rather than strict priority and acts only at the contending radios, so the benefit of the priority queue will be smaller than here. The results should be read as a comparison of mechanisms under controlled assumptions, not as predicted performance of a hospital network.

2) Baselines. The TCP baseline is a standard Reno-style sender with SACK recovery, a 200 ms minimum retransmission timeout and one connection per patient, with no tuning for thin streams, no tail-loss probes and no active queue management. Tuned TCP variants, separate connections per class, QUIC streams [4] CUBIC [7] or RACK-TLP [6] would likely narrow the gap and are not evaluated here. The buffer-size experiment indicates that much of TCP's penalty comes from deep buffers.

3) Assumptions about applications. Message sizes, rates and deadlines (Table I) are design assumptions. Encryption, authentication and transport-layer security, which add per-packet cost and are mandatory in clinical deployments, are not modelled. The transport does not provide congestion control for telemetry itself; it relies on telemetry being a small, bounded share of capacity, and the $N = 120$ result shows where this assumption starts to fail.

VI. CONCLUSION

This paper analysed an ns-3 baseline for healthcare telemetry on a congested 802.11g network, validated a Python replica of it, and evaluated a priority- and deadline-aware transport

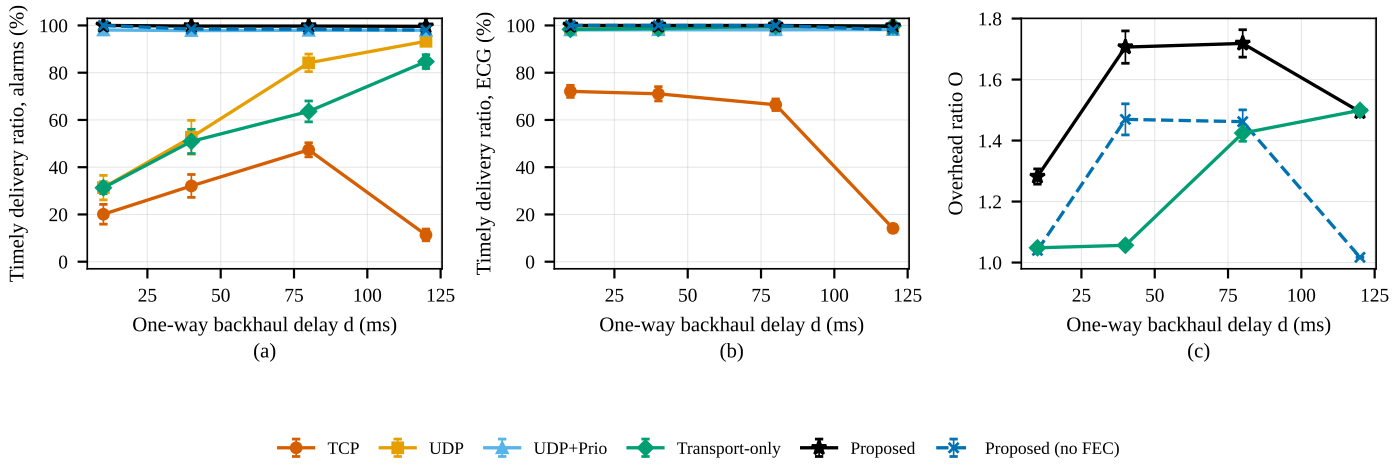


Fig. 8. Effect of one-way backhaul delay d ($N = 40$, $K = 8$; deadlines unchanged): (a) alarm and (b) ECG timely delivery ratio, and (c) overhead ratio of the proposed configuration with and without parity packets and alarm copies.

framework against TCP, UDP, UDP with priority queueing and a transport-only variant in a stress model with a shared bottleneck queue. The main findings are as follows. In the baseline ns-3 scenario TCP and UDP both deliver all healthcare packets with millisecond-scale delay at up to 8 Mbit/s of background load, because the congestion is confined to the background sender, and TCP's higher throughput is header overhead, so the baseline cannot distinguish transports. In the stress model, first, when bulk traffic keeps a deep bottleneck buffer full, TCP and UDP miss most alarm deadlines because of queueing delay, and TCP additionally suffers from head-of-line blocking: in the default scenario TCP delivered 20.1% of alarms on time. Second, priority queueing removes the queueing delay but cannot repair channel loss, whereas deadline-aware retransmission repairs loss but cannot help a message that is already late; only the combination reached about 100% timely delivery across 0–16 background flows, up to 100 patients and up to 55% bad-state loss. Third, parity packets and alarm copies are not free: at a 10 ms path delay they added about 24 points of overhead for no gain in timely delivery, and they paid off mainly when the round-trip time approached the deadline; without the congestion gate they made a heavily loaded link collapse. Fourth, the protection of telemetry has a cost for other users, 11% of background goodput in the default scenario, and strict priority can starve bulk traffic and the lowest telemetry class when telemetry load approaches capacity.

These conclusions hold for the simulated network and traffic model described in Section V, in which channel state is shared, the access point implements strict priority and the TCP baseline is untuned. The buffer-size experiment also shows that short buffers or active queue management remove a large part of the penalty for the baselines, so a deployment should consider them alongside any transport change.

Future work should validate the framework in a packet-level simulator with 802.11 enhanced distributed channel access and, ultimately, on a testbed with real monitoring devices; compare it with tuned TCP, QUIC and BBR; replace the retransmission timer with one whose margin scales with the round-trip time; enable redundancy on the ratio of round-trip time to deadline; add rate control or weighted scheduling for telemetry classes; account for transport security and the energy cost of retransmissions and parity on battery-powered wearables; and confirm the delivery deadlines with clinical stakeholders.

REFERENCES

[1] N. Samson, "healthcare-telemetry-ns3: TCP and UDP healthcare telemetry simulation in ns-3," GitHub repository (commit 1c07809), Oct. 2026. [Online]. Available: `https://github.com/nikhilsamsonsuresh-sketch/healthcare-telemetry-ns3`

[2] "ns-3 network simulator" (release 3.47 used for the baseline experiment). [Online]. Available: `https://www.nsnam.org/`

[3] W. Eddy (Ed.), "Transmission Control Protocol (TCP)," RFC 9293 (STD 7), IETF, Aug. 2022. [Online]. Available: `https://www.rfc-editor.org/rfc/rfc9293`

[4] J. Iyengar and M. Thomson (Eds.), "QUIC: A UDP-based multiplexed and secure transport," RFC 9000, IETF, May 2021. [Online]. Available: `https://www.rfc-editor.org/rfc/rfc9000`

[5] J. Iyengar and I. Swett (Eds.), "QUIC loss detection and congestion control," RFC 9002, IETF, May 2021. [Online]. Available: `https://www.rfc-editor.org/rfc/rfc9002`

[6] Y. Cheng, N. Cardwell, N. Dukkipati, and P. Jha, "The RACK-TLP loss detection algorithm for TCP," RFC 8985, IETF, Feb. 2021. [Online]. Available: `https://www.rfc-editor.org/rfc/rfc8985`

[7] L. Xu, S. Ha, I. Rhee, V. Goel, and L. Eggert (Ed.), "CUBIC for fast and long-distance networks," RFC 9438, IETF, Aug. 2023. [Online]. Available: `https://www.rfc-editor.org/rfc/rfc9438`

[8] B. Briscoe (Ed.), K. De Schepper, M. Bagnulo, and G. White, "Low latency, low loss, and scalable throughput (L4S) Internet service: Architecture," RFC 9330, IETF, Jan. 2023. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9330

[9] D. Borsatti, W. Cerroni, F. Tonini, and C. Raffaelli, "From IoT to cloud: Applications and performance of the MQTT protocol," in *Proc. 22nd Int. Conf. Transparent Optical Networks (ICTON)*, Bari, Italy, 2020, pp. 1–4, doi: 10.1109/ICTON51198.2020.9203167. [Online]. Available: https://research.chalmers.se/en/publication/523631

[10] I. Ahmad, F. Shahid, I. Ahmad, J. Islam, K. N. Haque, and E. Harjula, "Adaptive lightweight security for performance efficiency in critical healthcare monitoring," arXiv:2406.03786, 2024 (rev. Dec. 2024). [Online]. Available: https://arxiv.org/abs/2406.03786

[11] J. Zhou, Y. Sun, and C. Tellambura, "Revolutionizing medical data transmission with IoMT: A comprehensive survey of wireless communication solutions and future directions," arXiv:2504.02446, Apr. 2025. [Online]. Available: https://arxiv.org/abs/2504.02446

[12] N. Nomikos, T. Charalambous, R. Wichman, Y.-A. Pignolet, and N. Pappas, "On the interplay between deadline-constrained traffic and the number of allowed retransmissions in random access networks," *Entropy*, vol. 26, no. 8, Art. 655, Jul. 2024, doi: 10.3390/e26080655. [Online]. Available: https://www.mdpi.com/1099-4300/26/8/655

[13] J. Livingood, "Improving latency with active queue management (AQM) during COVID-19," arXiv:2107.13968, 2021 (rev. Feb. 2022). [Online]. Available: https://arxiv.org/abs/2107.13968

[14] S. Aoyagi, Y. Horie, K. Nguyen, and H. Sekiya, "An evaluation of active queue management and TCP congestion control in WiFi networks," in *Proc. MITA*, Jul. 2022, pp. 134–137. [Online]. Available: https://www.s-lab.nd.chiba-u.jp/achievements/abst.php?paper=683

[15] V. Sitaram A., V. T. G., A. George, R. Manivasakan, and B. Dappuri, "Performance evaluation of the QoS provisioning ability of IEEE 802.11e WLAN standard for multimedia traffic," arXiv:2112.07170, Dec. 2021. [Online]. Available: https://arxiv.org/abs/2112.07170

[16] Memoona and S. W. Kim, "Performance and simulation analysis of 802.11ax OFDMA in contention-driven scenarios," *PeerJ Comput. Sci.*, vol. 11, Art. e2687, 2025, doi: 10.7717/peerj-cs.2687. [Online]. Available: https://peerj.com/articles/cs-2687